# Solving the RCMPSP using ACO and SMT-Solver based Local Search

Alejandro Fernández Gil
*Departamento de Informática*
*Universidad Técnica Federico Santa María*
Valparaíso, Chile
affernan@jp.inf.utfsm.cl

Mariam Gómez Sánchez
*Departamento de Informática*
*Universidad Técnica Federico Santa María*
Valparaíso, Chile
mggomez@jp.inf.utfsm.cl

Carlos Castro
*Departamento de Informática*
*Universidad Técnica Federico Santa María*
Valparaíso, Chile
Carlos.Castro@inf.utfsm.cl

***Abstract*—The Resource-Constrained Project Scheduling Problem (RCPSP) is a classical well-known hard combinatorial scheduling problem in Operations Research. RCPSP has been widely tackled when implementing automated production systems and achieve an optimal assignment of activities to machines and reduction of total completion time.**

**An extension of the RCPSP is a Resource-Constrained Multi-Project Scheduling Problem (RCMPSP) with global and local resource constraints, based on multi-project, causing increased complexity in scheduling process.**

**We are interested in solving RCMPSP through a hybrid approach that integrates Ant Colony Optimization and a Local Search procedure based on Hill-Climbing First Improvement supported by an SMT Solver as a movement satisfaction verifier. Our approach was tested with a selected set of instances belong to the MPSPLIB library. We compare our approach with others of the state-of-the-art approaches and find that our approach is competitive in several instances against others.**

## I. Introduction

The Resource-Constrained Project Scheduling Problem (RCPSP) arises in the decade of 1960 [1], consists of the allocation of processing start times of various activities that are part of a project where exist constraints on the availability of resources and these activities must follow a strict order of precedence. Pritsker et al.[2] addresses the Resource-Constrained Multi-Project Scheduling Problem (RCMPSP), where several projects must be executed simultaneously competing for the use of global resources; later the possibility is added that the projects can be initiated at different times. Confessore et al. [3] propose the Decentral Resource Constrained Multi-Project Scheduling Problem (DRCMPSP) with the particularity that each project has its own set of renewable resources. DRCMPSP proposes the existence of an intelligent agent capable of making decisions for each project, and this agent manages global resources and does not know the use of own resources in each project. The objective pursued in the problems described above is based on finding a feasible schedule for a set of activities in each project, minimizing mainly the total time of completion of all activities (`Makespan`) added to the time of delay (`Delay`) in each project.

In this work we focus on solving DRCMPSP, without taking into account the decentralized part of the problem, having only a decision maker who is responsible for achieving a better overall performance, what is equivalent to solving an RCMPSP with global and local resource constraints. We propose a hybrid approach based on Ant Colony Optimization (ACO)[4] which provides feasible solutions, and on these solutions we apply a Local Search procedure based on Hill-Climbing (HC) First Improvement in order to improve the quality of these. As part of HC, we use Z3 Theorem Prover[5], a SMT solver to support the process of verifying movement satisfaction and precedence of activities, formalizing the restrictions as logical formulas so that a satisfactory solution expresses whether it is suitable or not perform the movement. We have obtained near-optimal solutions for several instances of the problem.

## II. Resource-Constrained Multi-Project Scheduling Problem

In the RCMPSP with global and local resource constraints exist a set of $\mathcal{P}$ projects that are scheduled simultaneously. Each $p$ project has a set of $A_p$ activities and a set of local resources $R_p$ to meet the demands of the activities belonging to $A_p$. Each $a$ activity of a $p$ project has a duration of $d_{ap}$, a set of successor activities $S_{ap}$ and a set of resources necessary for its execution $r_{ap}$. Until the end of the execution of the $a$ activity of the $p$ project, activities belonging to $S_{ap}$ cannot begin to be executed. For the start of the execution of the $a$ activity of the $p$ project, all resources $r_{ap}$ must be available simultaneously in the defined quantities and this availability must remain uninterrupted until the end of the execution. There is a set of global resources $\mathcal{G}$ shared by all projects. All global and local resources are considered renewable and once an activity is processed the occupied resources are released.

A solution to the problem consists in assigning the start times of the execution of each activity $a$ of each project $p$, so that the restrictions of precedence between the activities are fulfilled and that the necessary resources are available without interruption. In this research we seek to minimize the average value of the `Makespan` added to the `Delay` of each project.

$$\texttt{Min} \quad \frac{1}{\mathcal{P}}\sum_{p=1}^{\mathcal{P}}(M_p + D_p) \tag{1}$$

where $M_p$ and $D_p$ are the `Makespan` and `Delay` respectively associated with a $p$ project.

**Algorithm 1** General Procedure

```
Initialization_of_Parameters
S_F = ∅
while (N_ITER > 0) do
    M' = M
    S_Ant = ∅
    for i = 1 ... N_ANTS do
        sol = Constructive_Algorithm
        if f(sol) < f(S_Ant) or S_Ant.size = 0 then
            S_Ant = sol
        end if
    end for
    S_Ant = Local_Search(S_Ant)                ▷ HC, SMT Solver
    Evaporation_of_Pheromone_Levels in M
    if f(S_Ant) < f(S_F) then
        S_F = S_Ant
    end if
end while
return S_F
```

**Algorithm 2** Constructive_Algorithm (ACO)

```
sol = ∅
for i = 1 ... N_STEPS do
    F = Check_constraints
    a_i p_j = Obtain (sol)
    for a_w p_x in F do
        Find T_{a_i p_j, a_w p_x} in M'
        Calculate η_{a_w p_x}
        Calculate P_{a_i p_j, a_w p_x}
    end for
    Select node to realize movement
    Add the selected node to sol
    Update resource availability
end for
Deposit pheromone in M for movements involved in sol
return sol
```

**Algorithm 3** Local_Search($\mathcal{S}_{Ant}$)

```
for i = 1 ... N_ITERB do
    decision = UNSAT
    while decision = UNSAT do
        S'_Ant = Randomized_Movement(S_Ant)
        Φ = Boolean_Formulation(S'_Ant)
        decision = Check(Φ)
        if decision = SAT then
            S'_Ant = Recalculate_times(S'_Ant)
            if f(S'_Ant) < f(S_Ant) then
                S_Ant = S'_Ant
            end if
        end if
    end while
end for
Deposit pheromone in M for movements involved in S_Ant
return S_Ant
```

Fig. 1: Algorithms 1, 2, and 3 of our approach.

## III. Proposal

Our proposal is founded on hybrid approach based in ACO and HC First Improvement. The construction of a solution consists of crossing a graph where each node $a_i p_j$ represents a $i$ activity of a $j$ project, and an edge $a_i p_j, a_w p_x$ represents a transition of succession between the $i$ activity of the $j$ project towards the $k$ activity of the $p$ project. Each edge $a_i p_j, a_w p_x$ has an associated pheromone trail $\tau_{a_i p_j, a_w p_x}$ stored in the pheromone matrix $\mathcal{M}$. Each node has an associated heuristic preference $\eta_{a_i p_j}$ defined by:

$$\eta_{a_i p_j} = \frac{1}{d_{a_i p_j}} + \frac{1}{MPMT_{p_j}} \tag{2}$$

where $MPMT_{p_j}$ is the time to complete the $p_j$ project to which the $a_i$ activity belongs (see Algorithm 1 in Figure 1).

To represent solutions we have used a triad vector composed of an activity that belongs to a particular project, the project and the time it was located, as shown below:

$$sol = \{ (a_1, p_j, t_1) \rightarrow (a_2, p_j, t_2) \rightarrow ... \rightarrow (a_{|A_p|}, p_{|P|}, t_M) \}$$

### A. ACO algorithm

To construct a solution the ant must make $N_{STEPS}$ steps, which corresponds to the total activities of all projects. In each step the constraints are checked, obtaining a set $\mathcal{F}$ of nodes $a_l p_h$ towards which it is feasible to move, and the current node $a_i p_j$ where the ant is positioned is obtained.

For each element of $\mathcal{F}$, the probability $P_{a_i p_j, a_w p_x}$ of the node $a_w p_x$ is selected to make the transition $a_i p_j, a_w p_x$. Subsequently, following the Roulette-Wheel method [6], a node is selected to be stored in $sol$ along with the corresponding time and the amount of resources available for the next step is updated. The value of $P_{a_i p_j, a_w p_x}$ is calculated through the equation 3.

- $\alpha$ : influence of memorial information (pheromone trail)
- $\beta$ : influence of heuristic information

$$P_{a_i p_j, a_w p_x} = \frac{[\tau_{a_i p_j, a_w p_x}]^{\alpha} * [\eta_{a_w p_x}]^{\beta}}{\sum_{a_l p_h \in \mathcal{F}} [\tau_{a_i p_j, a_l p_h}]^{\alpha} * [\eta_{a_l p_h}]^{\beta}} \tag{3}$$

Once the solution of an ant is completed, pheromone is deposited on the path traveled according to the quality of the solution found (equation 5).

### B. Local Search algorithm

The Local Search is based on HC First Improvement. This algorithm works on feasible solutions and to verify the satisfacibility of movements we use the advantages of Z3 API in C++ (SMT Solver) (see LS Algorithm Figure 1).

When a movement is made within $S_{Ant}$, its feasibility is verified. This verification process is carried out with Z3, which checks if the precedence constraints are satisfied in a new order. If the verification get `UNSAT` then the selection process is repeated, but if the result get `SAT`, the start times of each $a_i p_j$ are recalculated in $\mathcal{S}_{Ant}$ considering the availability of resources.

### C. Movement Satisfaction Verification Process

A motivation for the verification process satisfiability of precedence between activities is efficient is the use of Z3 within the HC algorithm.

The SMT solvers are based on SAT, where given a set of logic formulas ($F = \{f_1, f_2, \ldots, f_n\}$) on Boolean variables ($x_1, x_2, \ldots, x_n$) related by logical connectives ($\neg$, $\wedge$, $\vee$), whose final objective is to determine what values must be assigned to the literals so that $\bigwedge_{i=1}^{3} f_i$ is $true$ [7].

We propose the following Boolean Constraints Formulation to verify the satisfaction of a movement.

- Parameter
  - $k_{ap}$ possible position to occupy in the memory of the ant for the $a$ activity of the $p$ project, $\forall_a \in \{1, \ldots, |A_p|\}$, $\forall_p \in \{1, \ldots, |\mathcal{P}|\}$
- Decision Variable
  - $J_{ap}$ occupied position in ant memory for activity $a$ of the project $p$, $\forall_a \in \{1, \ldots, |A_p|\}$, $\forall_p \in \{1, \ldots, |\mathcal{P}|\}$
- Precedence Constraints

$$\left( \bigwedge_{a=1}^{|\mathcal{P}|} \bigwedge_{p=1}^{|A_p|} \bigwedge_{q=1}^{|A_p|} J_{ap} < J_{aq} \right) \wedge \left( \bigwedge_{a=1}^{|\mathcal{P}|} \bigwedge_{p=1}^{|A_p|} J_{ap} = k_{ap} \right) \tag{4}$$

$$\forall J_{aq} \in S_{ap}$$

### D. Variation of Pheromone levels

The pheromone deposit is made on the edges composed of $a_i p_j, a_w p_x$ provided that $a_i p_j$ and $a_w p_x$ are in $sol$ and positioned consecutively $(i, i+1)$. The amount of pheromone to deposit depends on the quality of $sol$ and is calculated using the equation:

$$\tau_{a_i p_j, a_w p_x} \leftarrow \tau_{a_i p_j, a_w p_x} + \frac{1}{f(sol)} \; \forall (a_i p_j)_i, (a_w p_x)_{i+1} \in sol \tag{5}$$

The evaporation of pheromones is done on all edges $a_i p_j, a_w p_x$ of $\mathcal{M}$ through the following equation:

$$\tau_{a_i p_j, a_w p_x} \leftarrow (1-\rho) * \tau_{a_i p_j, a_w p_x} \; \forall a_i p_j, a_w p_x \in \mathcal{M} \tag{6}$$

In addition, with the objective of avoiding stucks in local optimal pheromone levels were limited between 0 and 1.5.

## IV. Computational Results

The algorithms were implemented C++ language and tested on a processor Intel(R) Xeon(R) CPU E3-1220 3.10 GHz with 32GB RAM memory on CentOS Linux 7.6.18.19. We validated our approach on a selected set of instances of MPSPLIB[1] datasets proposed in [8] and we made 100 run for each of the instances.

We compare our approach with the results of the state-of-art. Our results demonstrate that our approach is competitive with respect to other well-known approaches, evidenced by the ADP, TMS and DPD metrics. Taking into account that MPSPLIB presents solutions of centralized and decentralized algorithms, we to compare the quality of our solutions use the solutions proposed in the literature by Homberger in [9] with RES approach and Villafáñez et al.[10] with PSGSMINSLK approach, which present the best known results and fundamentally do not take into account the decentralized aspect of the problem.

The Table I columns 2-10 show the values for the previously described metrics found by the three approaches. It is shown for a reduced set of selected instances, the TMS values of our approach improve and matches some TMS values.

TABLE I: PERFORMANCE COMPARISON

| | PSGSMINSLK[10] | | | RES[9] | | | Our approach | | |
|---|---|---|---|---|---|---|---|---|---|
| Instance | APD | TMS | DPD | APD | TMS | DPD | APD | TMS | DPD |
| mp_j30_a2_nr4 | 14.5 | 58 | 9.19 | 14.0 | 59 | 5.65 | 13.0 | **56** | 8.48 |
| mp_j30_a5_nr2 | 17.8 | 80 | 12.0 | 16.8 | 84 | 13.31 | 18.0 | **79** | 12.40 |
| mp_j90_a10_nr4 | 2.1 | **150** | 2.33 | 5.1 | **150** | 6.36 | 6.2 | **150** | 4.87 |
| mp_j90_a20_nr2 | 10.0 | 168 | 8.84 | 8.2 | 174 | 9.25 | 9.6 | **167** | 8.22 |
| mp_j120_a5_nr2 | 40.0 | 181 | 37.14 | 30.2 | **178** | 34.71 | 38.6 | **178** | 35.54 |

The Figure 2 shows a solution obtained by our approach for instance $mp_j30_a5_nr2$ where a feasible planning of the activities is carried considering adequate consumption of global and local resources and the conformity of precedence relationships between the activities of each project.

[1]http://www.mpsplib.com/ Collection of multi-project instances (MPSPLIB)

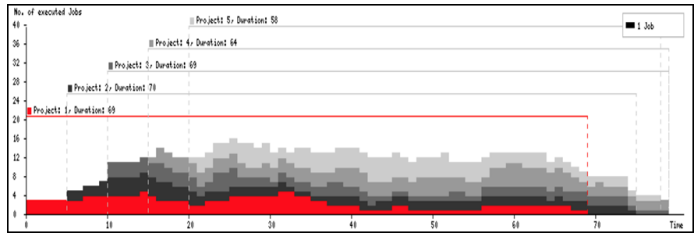


Fig. 2: Feasible planning of activities of each project.

## V. Conclusion

In this work we proposed a hybrid optimization approach that integrates Ant Colony Optimization and a Local Search based on a Hill-Climbing First Improvement algorithm supported in Z3 to solve RCMPSP. The computational results addressing the RCMPSP proved that the proposed approach was able to find better or equal solutions for some of the selected instances with respect to those known in MPSPLIB. The use of ACO proves to be a feasible resolution method to cover RCMPSP. An additional Local Search algorithm based on HC and based on Z3 was used, achieving improved solutions due to the efficiency of the process of satisfying precedence between activities in the movements.

Finally, as future work, it is essential to test our approach with the remaining MPSPLIB instances and try to improve the existing results to the centralized problem. In addition, we want to model RCMPSP using the advantages offered by the Satisfiability Modulo Theories like Z3 based on approaches, techniques and models such as those proposed by [11].

## References


[1] E. Davis and J. Patterson, "An exact algorithm for the multiple constrained project scheduling problem," *Yale University*, 1969.

[2] A. A. B. Pritsker, L. J. Waiters, and P. M. Wolfe, "Multiproject scheduling with limited resources: A zero-one programming approach," *Management science*, vol. 16, no. 1, pp. 93–108, 1969.

[3] G. Confessore, S. Giordani, and S. Rismondo, "A market-based multi-agent system model for decentralized multi-project scheduling," *Annals of Operations Research*, vol. 150, no. 1, pp. 115–135, 2007.

[4] M. Dorigo and M. Birattari, *Ant colony optimization*. Springer, 2010.

[5] L. De Moura and N. Bjørner, "Z3: An efficient smt solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.

[6] A. Lipowski and D. Lipowska, "Roulette-wheel selection via stochastic acceptance," *Physica A: Statistical Mechanics and its Applications*, vol. 391, no. 6, pp. 2193–2196, 2012.

[7] N. G. Ramirez, "A framework for autonomous generation of strategies in satisfiability modulo theories," Ph.D. dissertation, 2018.

[8] J. Homberger, "A ($\mu$, $\lambda$)-coordination mechanism for agent-based multi-project scheduling," *OR spectrum*, vol. 34, no. 1, pp. 107–132, 2012.

[9] ——, "A multi-agent system for the decentralized resource-constrained multi-project scheduling problem," *International Transactions in Operational Research*, vol. 14, no. 6, pp. 565–589, 2007.

[10] F. Villafáñez, D. Poza, A. López-Paredes, J. Pajares, and R. del Olmo, "A generic heuristic for multi-project scheduling problems with global and local resource constraints (rcmpsp)," *Soft Computing*, vol. 23, no. 10, pp. 3465–3479, 2019.

[11] J. Suy Franch *et al.*, "A satisfiability modulo theories approach to constraint programming," 2012.